



---

# Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

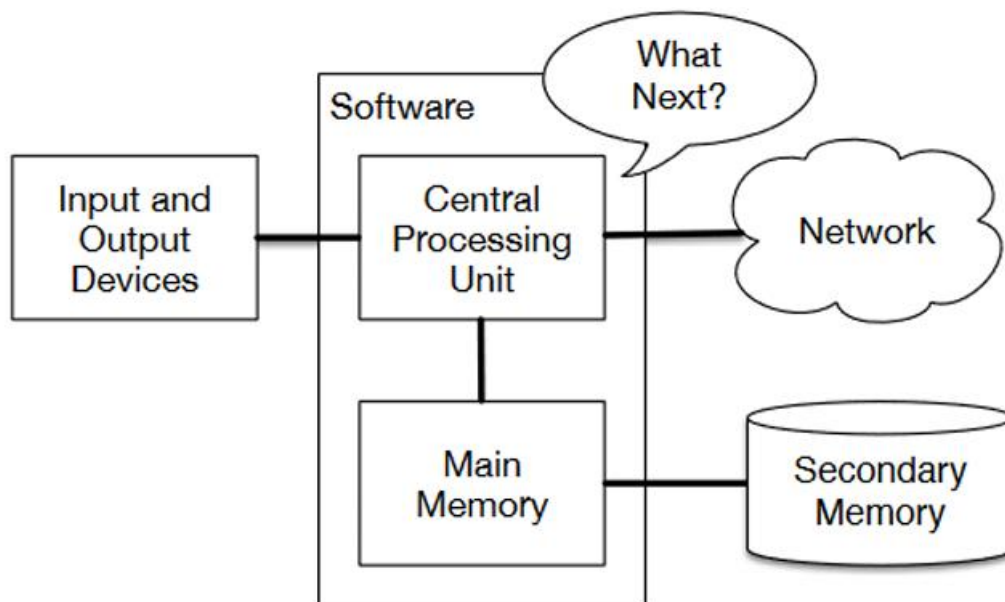
|                    |                      |
|--------------------|----------------------|
| NIM                | 71251176             |
| Nama Lengkap       | David Alexsi Cahyadi |
| Minggu ke / Materi | 10 / Manajemen File  |

PROGRAM STUDI INFORMATIKA  
FAKULTAS TEKNOLOGI INFORMASI  
UNIVERSITAS KRISTEN DUTA WACANA  
YOGYAKARTA  
2026

## BAGIAN 1: MATERI MINGGU INI (40%)

### Pengantar File

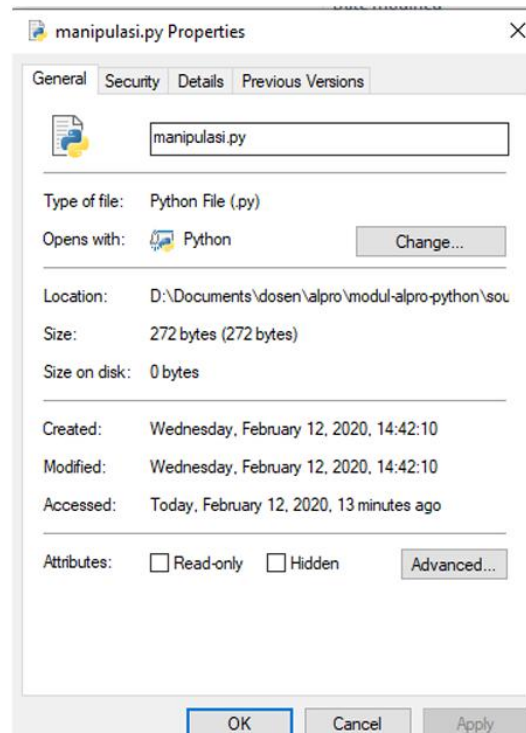
Setiap program yang sedang berjalan membutuhkan **memory primer** (RAM) sebagai tempat menyimpan data sementara. Namun, sifat dari memory primer ini adalah *volatile*, artinya semua data yang tersimpan di dalamnya akan hilang begitu program ditutup atau komputer dimatikan. Oleh karena itu, untuk menyimpan data secara permanen, dibutuhkan media penyimpanan yang berbeda, yaitu **secondary memory**.



Gambar 9.1 : Secondary Memory di Komputer, Sumber : <https://shorturl.at/Hq1OF>

Secondary memory mencakup perangkat seperti hard disk, SSD, flash drive, dan sejenisnya. File disimpan pada secondary memory sehingga data tidak akan hilang meskipun komputer dimatikan. Secara teknis, file adalah kumpulan bit data yang disimpan secara permanen di secondary memory, berupa sekumpulan informasi yang saling berelasi sebagai satu kesatuan.

File sendiri dapat berupa beberapa jenis, antara lain file sistem, file program (binary), file multimedia, hingga file teks. Setiap file memiliki properti tersendiri seperti nama file, ukuran, lokasi di hard disk, owner, hak akses, dan tanggal akses.

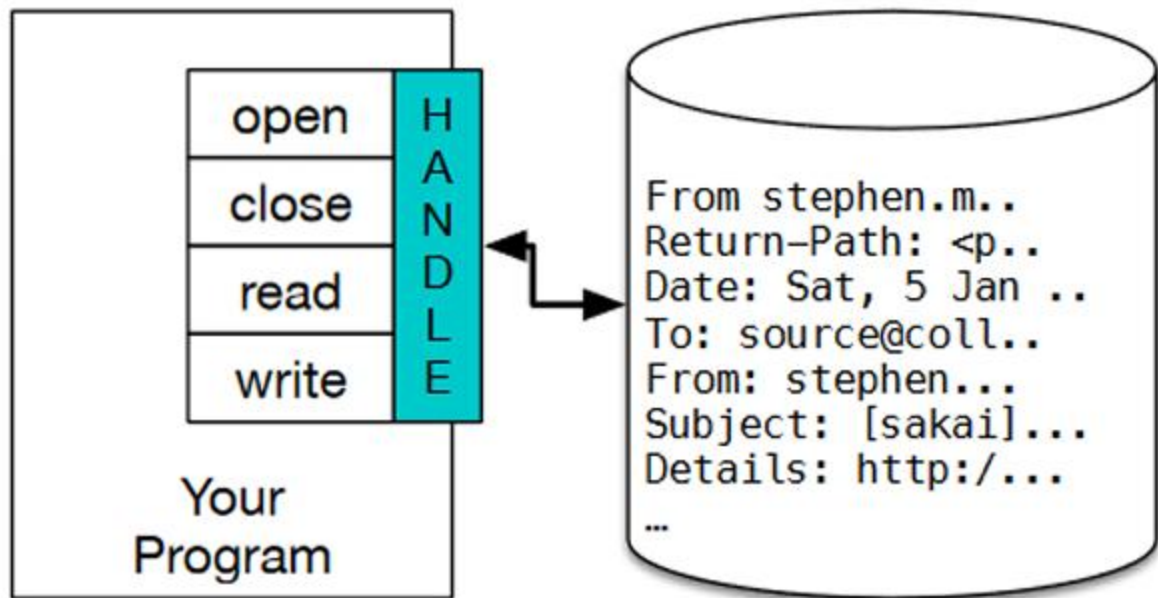


Gambar 9.2 : Contoh Property File, Sumber : <https://shorturl.at/Hq1OF>

## Pengaksesan File

Untuk dapat mengakses sebuah file dalam Python, terdapat alur langkah yang harus diikuti secara berurutan, yaitu:

1. Menyiapkan file dan path yang akan diakses
2. Open file
3. Lakukan sesuatu dengan file tersebut, seperti ditampilkan (read) isinya atau diubah / ditulisi (write)
4. Close file



Gambar 9.3 : Ilustrasi Handle File, Sumber : <https://shorturl.at/Hq1OF>

Dalam Python, file dibuka menggunakan fungsi **open()**. Fungsi ini **mengembalikan sebuah file handle**, yaitu objek yang digunakan sebagai “jembatan” antara program dengan file yang ada di disk. Sebagai contoh:

```
1 handle = open('mbox-short.txt')
2 print(handle)
```

Output yang dihasilkan berupa informasi nama file, mode akses (**r** = read), dan encoding yang digunakan (UTF-8). Jika file yang dimaksud tidak ditemukan, Python akan melempar error **FileNotFoundError**.

Pada file teks, isi file terdiri dari baris-baris string. Oleh karena itu, pembacaan file teks umumnya dilakukan baris demi baris hingga mencapai EOF (End of File).

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
Return-Path: <postmaster@collab.sakaiproject.org>
Date: Sat, 5 Jan 2008 09:12:18 -0500
To: source@collab.sakaiproject.org
From: stephen.marquard@uct.ac.za
Subject: [sakai] svn commit: r39772 - content/branches/
Details: http://source.sakaiproject.org/viewsvn/?view=rev&rev=39772
```

Gambar 9.4 : Contoh File Teks dan Barisnya, Sumber : <https://shorturl.at/Hq1OF>

## Manipulasi File

Setelah file berhasil dibuka, kita dapat melakukan berbagai manipulasi terhadap isinya. Cara dasar membaca isi file di Python adalah dengan melakukan **perulangan (loop)** pada setiap baris file. Pendekatan ini sangat dianjurkan karena lebih hemat memory dibandingkan membaca seluruh isi file sekaligus menggunakan **read()**, terutama untuk file berukuran besar yang berpotensi menyebabkan program hang atau crash.

---

```
1     handle = open('mbox-short.txt')
2     count = 0
3     for line in handle:
4         count = count + 1
5     print('Line Count:', count)
```

---

Selama proses perulangan, kita juga dapat menyaring baris-baris tertentu sesuai kebutuhan. Misalnya, untuk menampilkan hanya baris yang diawali kata **“Date:”**, digunakan metode **startswith()**, dan untuk mencari string tertentu di dalam baris, digunakan metode **find()**.

---

```
1     handle = open('mbox-short.txt')
2     count = 1
3     for line in handle:
4         if line.startswith("Date:") and count <= 10:
5             count += 1
6             print(line)
```

---

Perlu diperhatikan bahwa setiap baris pada file teks sudah mengandung karakter *newline* ( `\n` ) di ujungnya. Apabila fungsi **print()** digunakan tanpa modifikasi, akan terjadi baris kosong pada output. Untuk mengatasinya, dapat digunakan metode **rstrip()** pada setiap baris sebelum dicetak.

## Penyimpanan File

Selain membaca, Python juga memungkinkan kita untuk **menulis data ke dalam file**. Caranya hampir sama dengan membuka file untuk dibaca, namun mode yang digunakan diubah dari **'r'** (read) menjadi **'w'** (write).

---

```
1     handle = open('output.txt','w')
2     tulisan = "teks ini akan dituliskan ke file\n"
3     handle.write(tulisan)
4     handle.close()
```

---

Setelah file dibuka dengan mode **'w'**, penulisan isi dilakukan menggunakan metode **write()** yang menerima argumen berupa string. Setelah selesai menulis, file **wajib ditutup** menggunakan **close()** untuk memastikan semua data tersimpan dengan benar dan resource sistem dilepaskan. Apabila file dengan nama yang diberikan belum ada, Python akan membuatnya secara otomatis. Namun jika sudah ada, isi file sebelumnya akan ditimpa (overwrite).

## BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Github:[https://github.com/DavidCahyadi1013/71251176\\_David-Alexsi-Cahyadi.git](https://github.com/DavidCahyadi1013/71251176_David-Alexsi-Cahyadi.git)

### Latihan 9.1

#### Soal:

Latihan 9.1 Buatlah sebuah program yang dapat membandingkan 2 buah file teks dan kemudian menampilkan perbedaan antar kedua teks per barisnya jika ada perbedaan!

#### Source Code:

```
def isi (nama):
    with open(nama, encoding='utf-8') as f:
        konten = f.readlines()
    return konten

def beda(teks1,teks2):
    isi1 = isi(teks1)
    isi2 = isi(teks2)
    total = max(len(isi1), len(isi2))
    print (f"File 1 : {len(isi1)} baris")
    print (f"File 2 : {len(isi2)} baris")
    ketemu= False
    for i in range (total):
        if i < len (isi1):
            baris1 = isi1[i].rstrip()
        else:
            baris1 = "Baris Tidak Ada"
        if i < len(isi2):
            baris2 = isi2[i].rstrip()
        else:
            baris2 = "Baris Tidak Ada"
        if baris1 != baris2:
            ketemu = True
            print(f"Baris {i+1} berbeda:")
            print(f" < {teks1}: {baris1}")
```

```

        print(f" > {teks2}: {baris2}")
        print()
    if not ketemu:
        print("Kedua file sama persis.")

if __name__ == "__main__":
    try:
        teks1 = input("Nama File pertama: ")
        teks2 = input("Nama File kedua: ")
        beda(teks1, teks2)
    except FileNotFoundError as e:
        print(f"File Tidak Ditemukan: {e}")

```

## Output:

```

1 def isi (nama):
2     with open(nama, encoding='utf-8') as f:
3         konten = f.readlines()
4         return konten
5
6 def beda(teks1, teks2):
7     isi1 = isi(teks1)
8     isi2 = isi(teks2)
9     total = max(len(isi1), len(isi2))
10    print ("File 1 : (len(isi1)) baris")
11    print ("File 2 : (len(isi2)) baris")
12    ketemu= False
13    for i in range (total):
14        if i < len (isi1):
15            baris1 = isi1[i].rstrip()
16        else:
17            baris1 = "Baris Tidak Ada"
18        if i < len(isi2):
19            baris2 = isi2[i].rstrip()

```

PS D:\Pralpro\Pertemuan 10> & C:\Users\dvda1\AppData\Local\Programs\Python\Python313\python.exe "d:\Pralpro\Pertemuan 10\soal01.py"  
 Nama File pertama: sampel1.txt  
 Nama File kedua: sampel2.txt  
 File 1 : 60 baris  
 File 2 : 60 baris  
 Baris 1 berbeda:  
 < sampel1.txt: Bahasa pemrograman Java pertama kali dikembangkan oleh James Gosling.  
 > sampel2.txt: Bahasa pemrograman Python pertama kali dikembangkan oleh Guido van Rossum.  
 Baris 2 berbeda:  
 < sampel1.txt: Java dirilis secara resmi oleh Sun Microsystems pada tahun 1995.  
 > sampel2.txt: Python dirilis secara resmi pada tahun 1991 sebagai bahasa open source.  
 Baris 3 berbeda:  
 < sampel1.txt: Java merupakan bahasa pemrograman yang bersifat berorientasi objek.  
 > sampel2.txt: Python merupakan bahasa pemrograman tingkat tinggi yang mudah dipelajari.

```

Baris 57 berbeda:
< sampel1.txt: Java ME digunakan untuk pengembangan aplikasi pada perangkat mobile lama.
> sampel2.txt: Python digunakan secara luas di bidang otomatisasi dan scripting sistem.

Baris 58 berbeda:
< sampel1.txt: Performa Java relatif lebih lambat dibandingkan bahasa C dan C++.
> sampel2.txt: Performa Python relatif lebih lambat dibandingkan bahasa C dan Java.

Baris 59 berbeda:
< sampel1.txt: Namun Java tetap populer karena kemudahan pengelolaan dan portabilitasnya.
> sampel2.txt: Namun Python tetap populer karena kemudahan penulisan dan ekosistem librarynya.

Baris 60 berbeda:
< sampel1.txt: Java hingga saat ini masih menjadi salah satu bahasa paling banyak digunakan di dunia.
> sampel2.txt: Python hingga saat ini menjadi salah satu bahasa pemrograman paling populer di dunia.

```



### Penjelasan:

Program diawali dengan mendefinisikan fungsi **isi(nama)** yang bertugas untuk membaca isi file teks. Fungsi ini membuka file dengan encoding UTF-8 menggunakan perintah **open**, kemudian membaca seluruh isi file baris per baris dan menyimpannya ke dalam sebuah list. Setiap elemen dalam list tersebut merepresentasikan satu baris dari file. Setelah itu, list dikembalikan agar dapat digunakan oleh bagian program lainnya.

Selanjutnya, program memiliki fungsi utama yaitu **beda(teks1, teks2)** yang digunakan untuk membandingkan dua file teks. Pada awal fungsi ini, kedua file dibaca menggunakan fungsi **isi**, sehingga diperoleh dua list yang berisi baris-baris dari masing-masing file. Program kemudian menentukan jumlah iterasi menggunakan nilai maksimum dari panjang kedua list, sehingga seluruh baris dari kedua file tetap dapat dibandingkan meskipun jumlah barisnya berbeda. Informasi jumlah baris dari masing-masing file juga ditampilkan ke layar sebagai informasi awal.

Proses perbandingan dilakukan menggunakan perulangan **for** berdasarkan jumlah iterasi yang telah ditentukan. Pada setiap iterasi, program mengambil baris ke-*i* dari masing-masing file. Jika salah satu file tidak memiliki baris pada indeks tersebut (karena jumlah baris lebih sedikit), maka program akan menggantinya dengan teks "Baris Tidak Ada". Setiap baris kemudian dibersihkan dari karakter newline di akhir menggunakan metode **rstrip()** agar perbandingan lebih akurat. Setelah itu, kedua baris dibandingkan, jika terdapat perbedaan, program akan menandainya dan menampilkan nomor baris beserta isi dari kedua file yang berbeda tersebut.

Selama proses perbandingan, program menggunakan sebuah variabel penanda bernama **ketemu** untuk mencatat apakah ditemukan perbedaan atau tidak. Jika setidaknya satu perbedaan ditemukan, variabel ini akan bernilai benar. Setelah seluruh proses perulangan

selesai, program akan memeriksa nilai dari variabel tersebut. Jika tidak ada perbedaan yang ditemukan, maka program akan menampilkan pesan bahwa kedua file sama persis.

Bagian akhir, pada bagian utama program, pengguna diminta untuk memasukkan nama dua file yang akan dibandingkan. Pemanggilan fungsi **beda** dilakukan di dalam blok **try-except** untuk menangani kemungkinan kesalahan, seperti file yang tidak ditemukan. Jika terjadi kesalahan tersebut, program akan menampilkan pesan error yang sesuai sehingga pengguna menjadi tau file yang dimasukan tidak valid.

## Latihan 9.2

### Soal:

**Latihan 9.2** Buatlah sebuah program untuk menampilkan soal sederhana yang diambil dari file teks soal.txt yang memiliki format sebagai berikut:

```
1+1 = || 2
Bendera Indonesia? || Merah Putih
Kota gudeg adalah: || Yogyakarta
Komponen PC untuk penyimpanan file adalah... || harddisk
50 * 20 = || 1000
```

Dari soal tersebut tampilkan sbb:

```
nama file1: soal.txt
1+1 =
Jawab: 2
Jawaban benar!
Bendera Indonesia?
Jawab: merah putih
Jawaban benar!
Kota gudeg adalah:
Jawab: yogya
Jawaban salah!
Komponen PC untuk penyimpanan file adalah...
Jawab: HARDDISK
Jawaban benar!
```

### Source Code:

```
file = input("nama file: ")
handle = open(file)
for line in handle:
    line = line.strip()
    bagian = line.split("||")
    soal = bagian[0].strip()
```

```

    kunci = bagian[1].strip()
    print(soal)
    jawab = input("Jawab: ")
    if jawab.lower() == kunci.lower():
        print("Jawaban benar!")
    else:
        print("Jawaban salah!")
handle.close()

```

## Output:

```

PS D:\PrAipro\Pertemuan 10> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAipro/Pertemuan 10/soal02.py"
1+1 =
Jawab: 2
Jawaban benar!
Bendera Indonesia?
Jawab: merah putih
Jawaban benar!
Kota gudeg adalah:
Jawab: yogya
Jawaban salah!
Komponen PC untuk penyimpanan file adalah...
Jawab: HARDISK
Jawaban benar!
50 * 20 =
Jawab: 1000
Jawaban benar!

```

## Penjelasan:

Program dimulai dengan meminta pengguna memasukkan nama file melalui **input()**, lalu file dibuka menggunakan **open()**. Setelah file terbuka, program membaca isi file baris per baris menggunakan **for line in handle**.

Untuk setiap baris yang dibaca, program membersihkan spasi atau newline di awal dan akhir baris menggunakan **strip()**. Kemudian baris tersebut dipotong menjadi dua bagian menggunakan **split("|")** karena format file memisahkan soal dan jawaban dengan tanda **|**. Bagian pertama menjadi soal dan bagian kedua menjadi kunci jawaban.

Soal kemudian ditampilkan ke layar, lalu pengguna diminta memasukkan jawabannya lewat **input()**. Jawaban pengguna dibandingkan dengan kunci jawaban, keduanya diubah ke huruf kecil terlebih dahulu menggunakan **.lower()** agar perbandingan tidak sensitif terhadap huruf

besar atau kecil. Jika cocok maka ditampilkan “Jawaban benar!”, jika tidak maka “Jawaban salah!”. Proses ini berulang sampai semua baris di file habis, lalu file ditutup dengan **handle.close()**.